CHRISTOPHER NEWPORT UNIVERSITY · SCHOOL OF ENGINEERING & COMPUTING

Prompting LLMs with

Hugging Face

The three chat roles (system, user, assistant) and how to write precise, reliable prompts

Keith Perkins · School of Engineering & Computing

WHY IT MATTERS

Prompting is programming with language

> “Prompting is the bridge between what you want and what the model produces.”

Precise instructions in plain language replace traditional code syntax.

1

It controls behavior

The prompt sets the tone, structure, and quality of every response.

2

It comes first

Master prompting before agents, tools, or fine-tuning. They all build on it.

3

It's the cheapest lever

No training and no GPUs: change the text and run it again.

THE BUILDING BLOCKS

The three chat roles

Chat models organize every conversation into messages, and each message has a role.

`"system"`

System

Sets the model's identity, rules, and limits: tone, expertise level, and guardrails. Written by you, the developer.

`"user"`

User

The actual request or question, plus any context the model needs to do the task.

`"assistant"`

Assistant

The model's reply. You don't write it; the model generates it from the system and user messages.

Key insight: roles are a way of organizing prompts, not something built into the transformer. The model only ever sees one long sequence of tokens; the next slide shows how the roles get there.

What the model actually sees

YOUR MESSAGES (PYTHON)

```
messages = [
  {"role": "system",
   "content": "You are a clear CS tutor."},
  {"role": "user",
   "content": "Explain transformers
               in one sentence."},
]
```

AFTER THE CHAT TEMPLATE (QWEN 2.5 FORMAT)

```
<|im_start|>system
You are a clear CS tutor.<|im_end|>
<|im_start|>user
Explain transformers in one sentence.<|im_end|>
<|im_start|>assistant
▌ the model continues from here
```

The model just predicts the next token. Special tokens mark who is speaking, and the model was fine-tuned to answer after the assistant tag.

Anatomy of a complete prompt

STEP 1

Define behavior

The system message sets identity and constraints

STEP 2

Ask clearly

The user message states the exact task

STEP 3

Expect structure

The assistant fills in a response in the shape you asked for

EXAMPLE

```
system:     "You are a precise and helpful teaching assistant."
user:       "Explain attention in transformers in two bullet points."
assistant:  (the model fills this in)
```

Calling a hosted model with InferenceClient

PYTHON · HUGGINGFACE_HUB

```python
import os
from huggingface_hub import InferenceClient

client = InferenceClient(token=os.environ["HF_TOKEN"])

messages = [
  {"role": "system", "content": "You are a clear CS tutor."},
  {"role": "user", "content": "Explain transformers in one sentence."},
]

response = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=messages,
    temperature=0.2,
    max_tokens=500,
)
print(response.choices[0].message.content)
```

`messages`

The list of role + content dictionaries

`temperature`

Randomness. Near 0 = almost always the most likely word; higher = more varied

`max_tokens`

Caps the reply's length, which limits cost and waiting time

`HF_TOKEN`

Your access token from huggingface.co/settings/tokens. Hosted calls are metered

Or run a small model yourself, no account needed

PYTHON · TRANSFORMERS (RUNS ON COLAB, EVEN CPU)

```python
from transformers import pipeline

chat = pipeline("text-generation",
                model="Qwen/Qwen2.5-0.5B-Instruct")

messages = [
  {"role": "system", "content": "You are a clear CS tutor."},
  {"role": "user", "content": "Explain transformers in one sentence."},
]

out = chat(messages, max_new_tokens=100)
print(out[0]["generated_text"][-1]["content"])

# generated_text = the whole chat, reply appended last
```

Hosted (InferenceClient)

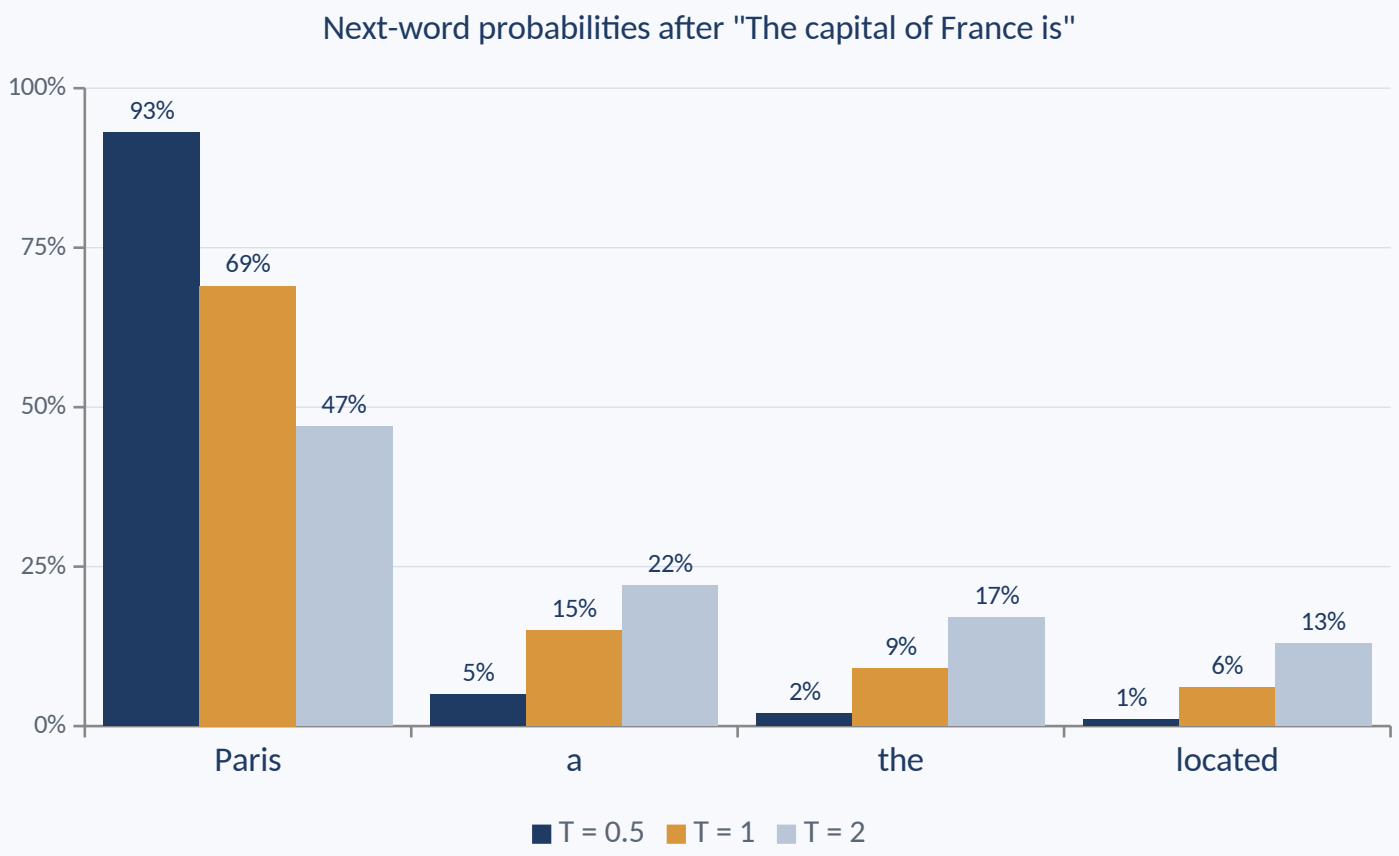
Huge models, fast, nothing to install. Needs a token; usage is metered.

Local (pipeline)

Free and private, and you can see every step. Small models make more mistakes.

A KEY SETTING

Temperature: how adventurous the model is



HOW IT WORKS

The model scores every possible next word. Temperature divides those scores before they become probabilities.

Low (≈ 0):

same answer every time. Use for facts, code, grading.

High (≥ 1):

more variety, more mistakes. Use for brainstorming.

BEST PRACTICES

Three habits of good prompts

1

Define a clear role

Start with "You are…" to set identity and expertise.

"You are a senior software architect specializing in distributed systems."

2

State explicit constraints

Set length, format, and style. Say what not to do.

"Answer in 3 bullets, under 20 words each. Don't speculate."

3

Iterate like you debug

Test variations, compare outputs, refine. Small wording changes can matter a lot.

Change one thing at a time and keep the prompts that work.

TECHNIQUE

Few-shot prompting: show, don't just tell

ZERO-SHOT: INSTRUCTIONS ONLY

```
system: Classify the review's sentiment.
user: It was fine, I guess.
```

The model must guess what labels you want and how to format them. You might get "Neutral", "Mixed", a paragraph, or "positive".

FEW-SHOT: ADD WORKED EXAMPLES

```
system: Classify sentiment as POSITIVE or NEGATIVE.
user: Loved every minute.
assistant: POSITIVE
user: Total waste of money.
assistant: NEGATIVE
user: It was fine, I guess.
assistant: (model answers in the same format)
```

The examples teach the labels and the format. Two to five examples are usually enough.

TECHNIQUE

Ask for output your code can parse

PYTHON

```
import json

system = ("Reply ONLY with JSON like "
  '{"sentiment": "positive" or "negative",'
  ' "confidence": a number from 0 to 1}')

reply = ask(system, "The plot dragged, but I loved the ending.")

try:
    result = json.loads(reply)
    print(result["sentiment"], result["confidence"])
except json.JSONDecodeError:
    print("Not valid JSON, retry or fix:", reply)
```

Show the exact shape

Give the keys and allowed values. An example JSON object works even better.

Always validate

Models usually comply but not always. Wrap parsing in try/except and handle failures.

Use structured-output features

Many APIs can force valid JSON (response_format). Use them when you can.

PUTTING IT TOGETHER

From bad to better: one prompt, three versions

1 Bad

Explain attention.

No context, no role, no constraints

2 Better

System: You are a CS tutor.
User: Explain attention in transformers.

Has a role, but no output constraints

3 Even better

System: You are a CS tutor for 4th-year college CS students.
User: Explain attention in transformers in 3 short bullets, under 20 words each.

Clear audience, explicit constraints, structured output

Try all three in the notebook and compare the outputs.

WHAT CHANGED?

- Added a system role that defines expertise and audience
- Specified the output format: 3 bullets
- Set a length limit: under 20 words each

KNOW THE LIMITS

A good prompt is not a guarantee

Made-up facts

Models can state false things confidently ("hallucination"). Verify anything that matters, especially citations and numbers.

Sensitivity to wording

Rephrasing a prompt, or reordering examples, can change the answer. Test prompts on several inputs, not one.

Prompt injection

Text from users or documents can override your instructions ("Ignore previous instructions…"). Never put secrets in a system prompt.

SUMMARY

Key takeaways

Three roles

System sets the rules, user asks, assistant answers. All of it becomes one token sequence.

Two ways to run

InferenceClient for big hosted models; pipeline for small local ones. Same messages list.

Be specific

Role, audience, format, and length. Show examples (few-shot) and ask for parseable output.

Test and verify

Iterate like you debug, keep temperature low for facts, and check what the model claims.

Think of prompting as programming with language: precise instructions get reliable results.